

Firmware Implementation	2
1. Title	2
2. Objective.....	2
3. Theory (Multi-Level).....	2
3.1 Beginner — Simple Intuition	2
3.2 Intermediate — Engineering View.....	2
3.3 Advanced — Signal/Architecture Level.....	3
4. Architecture / Concept.....	4
Firmware Build Flow.....	4
Application Logic (fw/main.c).....	5
Register Access in Assembly (Generated by Compiler)	5
5. Implementation.....	5
Files in This Module	5
Firmware Size Analysis	6
6. Lab / Hands-on Section.....	6
Step 1: Build Firmware.....	6
Step 2: Inspect the Compiled Firmware	7
Step 3: View ROM Hex Content	7
Step 4: Run SoC with this Firmware.....	7
Step 5: Verify Firmware is Loaded.....	7
7. Waveform / Verification Insight	7
Instruction Fetch Trace	7
UART TX Activity.....	8
Firmware String in ROM	8
8. Common Issues & Debug Tips.....	8
Verify RISC-V ABI with objdump	8
9. Summary	9

Firmware Implementation

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

Firmware Implementation — Writing Embedded C for PicoRV32, UART, and AXI-Lite SoC

2. Objective

By the end of this module, learners will be able to:

- Write bare-metal embedded C firmware for a RISC-V SoC with no OS
- Understand startup code (`crt0.S`) and linker scripts (`link.ld`)
- Implement UART driver functions using memory-mapped register access
- Compile firmware using `riscv32-unknown-elf-gcc` toolchain
- Convert firmware binary to ROM hex format for simulation and synthesis

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

When you write C code for a PC, the operating system provides everything: `printf()`, file access, memory management. When writing firmware for a bare-metal RISC-V SoC, there is **no OS**. You must:

1. Set up the stack yourself (where does the CPU put local variables?)
2. Write directly to hardware registers using fixed addresses
3. Build a tiny “stub” (`crt0.S`) that runs before `main()`

Think of it like writing a recipe where you also have to build the kitchen from scratch.

3.2 Intermediate — Engineering View

Bare-metal firmware components:

Component	File	Purpose
C runtime startup	<code>fw/crt0.S</code>	Initialize stack, call <code>main()</code> , trap on exit
Application code	<code>fw/main.c</code>	Application logic (UART echo)
Linker script	<code>fw/link.ld</code>	Map code/data to ROM/SRAM addresses
Firmware Makefile	<code>fw/Makefile</code>	Compile → link → objcopy → bin2hex

Component	File	Purpose
Hex converter	fw/bin2hex.py	Convert binary to 4-byte-wide hex for \$readmemh

Memory layout (from fw/link.ld):

```
MEMORY {
    ROM (rx) : ORIGIN = 0x00000000, LENGTH = 32K
    SRAM (rwx) : ORIGIN = 0x00010000, LENGTH = 8K
}
```

```
SECTIONS {
    .text → ROM (0x0000_0000): code + read-only data
    .data → SRAM (0x0001_0000): initialized global variables
    .bss → SRAM : zero-initialized globals
    Stack : top at 0x0001_00FC (descends)
}
```

UART register macros (fw/main.c):

```
#define UART_TX (*(volatile char*)0x10000000)
#define UART_RX (*(volatile char*)0x10000004)
#define UART_STATUS (*(volatile int*) 0x10000008)
```

volatile is mandatory — it prevents the compiler from caching the register read in a CPU register and bypassing the actual memory-mapped access.

3.3 Advanced — Signal/Architecture Level

Startup Sequence (crt0.S → main()):

```
# crt0.S: executed before main()
_start:
    # 1. Initialize stack pointer
    lui sp, %hi(__stack_top)
    addi sp, sp, %lo(__stack_top)
    # __stack_top = 0x0001_00FC (from link.ld)

    # 2. Zero-initialize BSS section
    la a0, __bss_start
    la a1, __bss_end
bss_loop:
    bgeu a0, a1, bss_done
    sw x0, 0(a0)
    addi a0, a0, 4
    j bss_loop
bss_done:

    # 3. Call main
    call main
```

```
# 4. Trap if main returns (should not happen)
_exit:
    j _exit
```

Compilation flags:

```
CFLAGS = -march=rv32i -mabi=ilp32 -nostartfiles -nostdlib
          -T link.ld -Wall -O2
```

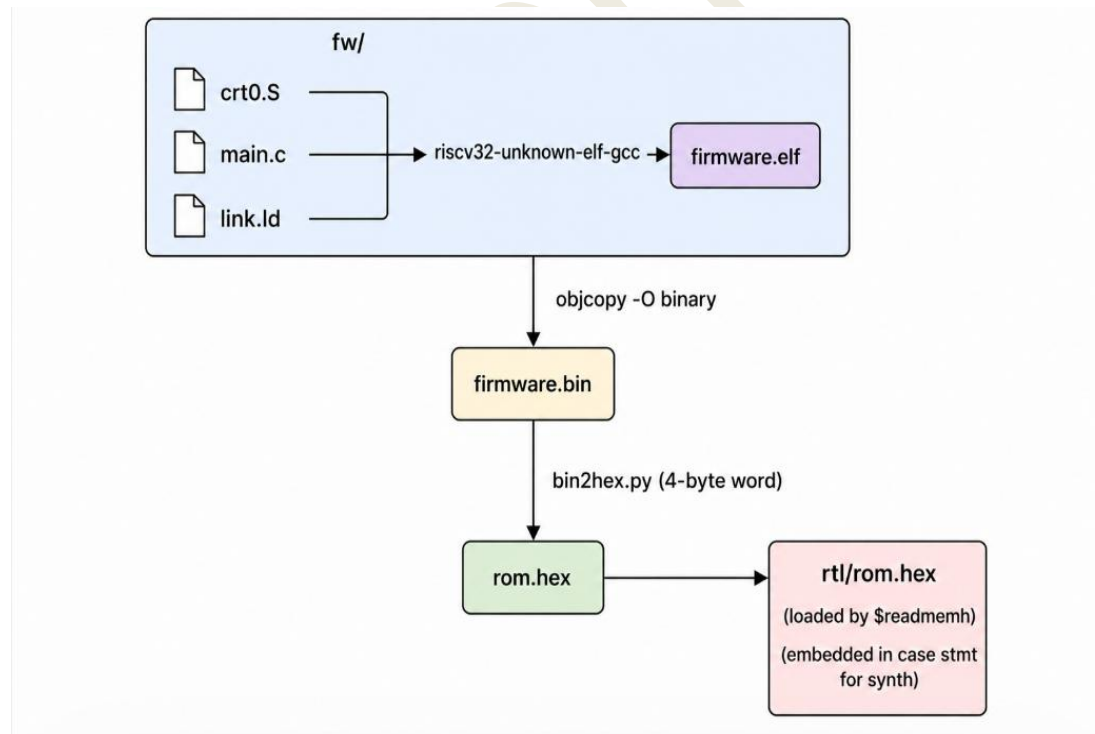
- `-march=rv32i`: Target RV32I base ISA (no multiply/float — matches `ENABLE_MUL=0` in PicoRV32)
- `-mabi=ilp32`: 32-bit integer, 32-bit pointers ABI
- `-nostartfiles`: Don't link GCC's `crt0` (we provide our own)
- `-nostdlib`: Don't link `libc` (we implement everything ourselves)

ROM hex format (rom.hex): Each line is one 32-bit little-endian instruction word in hex:

```
00020137 ← LUI  x2, 0x20 (first instruction: setup stack pointer upper)
FFC10113 ← ADDI x2, x2, -4
...
```

4. Architecture / Concept

Firmware Build Flow



Application Logic (fw/main.c)

```
main()
├── for i = 0..9:
│   ├── uart_puts("Hello Deepak from Nielit!\n")
│   │   ├── uart_putc(each char)
│   │   └── UART_TX = char [→ AXI write to 0x10000000]
└── while(1):
    ├── ← Echo loop
    ├── c = uart_getc()
    │   ├── while(!(UART_STATUS & 0x2)); [poll 0x10000008]
    │   └── return UART_RX [read 0x10000004]
    └── uart_putc(c)
        └── UART_TX = c [→ AXI write to 0x10000000]
```

Register Access in Assembly (Generated by Compiler)

For UART_TX = 'H' (C store through volatile pointer):

```
lui    a0, 0x10000    # a0 = 0x10000000
li     a1, 'H'        # a1 = 0x48
sb     a1, 0(a0)      # store byte → mem[0x10000000] = 0x48
                        # mem_wstrb = 0x1 (byte store)
```

For uart_getc() status poll:

```
lui    a0, 0x10000    # a0 = 0x10000000
lw     t0, 8(a0)      # t0 = mem[0x10000008] = STATUS
andi   t0, t0, 2      # check bit 1
beq    t0, x0, -8      # if not ready, loop
lb     a0, 4(a0)      # return mem[0x10000004] = RX byte
```

5. Implementation

Files in This Module

File	Purpose
fw/main.c	Application: greeting + echo loop (201 lines, well documented)
fw/crt0.S	Startup: stack init + call main + trap
fw/link.ld	Linker script: ROM at 0x0, SRAM at 0x10000, stack at 0x1_00FC
fw/Makefile	Build recipe: compile → link → binary → hex

File	Purpose
fw/bin2hex.py	Converts firmware binary to 4-byte hex words for \$readmemh
fw/gen_rom_case.py	Generates synthesizable ROM case-statement from rom.hex
fw/firmware.elf	Pre-built ELF (for inspection with objdump)
fw/firmware.bin	Raw binary (167 bytes = ~42 instructions)

Firmware Size Analysis

riscv32-unknown-elf-size fw/firmware.elf

Expected output:

```
text    data    bss     dec     hex filename
132      35       0     167     a7 firmware.elf
```

- .text = 132 bytes ≈ 33 instructions
- .data = 35 bytes = "Hello Deepak from Nielit!" string (26 chars + terminator × alignment)

6. Lab / Hands-on Section

Step 1: Build Firmware

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
make fw
```

This runs fw/Makefile:

```
# 1. Compile + Link
riscv32-unknown-elf-gcc \
    -march=rv32i -mabi=ilp32 \
    -nostartfiles -nostdlib \
    -T link.ld \
    -o firmware.elf crt0.S main.c
```

2. Create binary

```
riscv32-unknown-elf-objcopy -O binary firmware.elf firmware.bin
```

3. Generate hex for simulation

```
python3 bin2hex.py firmware.bin > ../rtl/rom.hex
```

4. Generate synthesizable ROM case statement

```
python3 gen_rom_case.py rom.hex > rom_case.v
```

Step 2: Inspect the Compiled Firmware

Disassemble the ELF

```
riscv32-unknown-elf-objdump -d fw/firmware.elf
```

Expected output (first lines):

Disassembly of section .text:

```
00000000 <_start>:
    0: 00020137    lui    sp,0x20          # sp = 0x00020000
    4: ffc10113    addi   sp,sp,-4         # sp = 0x0001FFFC (stack top)
    8: 00010297    auipc  t0,0x10
    ...
```

Step 3: View ROM Hex Content

```
cat rtl/rom.hex
```

```
00020137
FFC10113
00010297
FF828293
...
6C6C6548    ← ASCII 'lleH' (little-endian "Hell")
6544206F    ← ASCII 'eD o'
6B617065    ← ASCII 'kape'
...
```

Step 4: Run SoC with this Firmware

```
make soc_pico_uart_mem    # Uses picorv32_uart_mem sub-project
# OR
make soc                  # Uses AXI-based SoC design
```

Step 5: Verify Firmware is Loaded

Add to testbench (or check existing logs):

```
[ROM] loaded from plusarg: .../rtl/rom.hex
[ROM] Loaded 42 words
```

7. Verification Insight

Instruction Fetch Trace

In the SoC waveform (make wave), observe: 1. After reset, cpu_mem_addr = 0x0000_0000 (fetching instruction 0 = LUI sp, 0x20) 2. cpu_mem_addr = 0x0000_0004 (fetching instruction 1 = ADDI sp, sp, -4) 3. Sequential fetches until the loop at uart_puts call site

UART TX Activity

Look for clusters of `cpu_mem_addr = 0x1000_0000` writes with `mem_wstrb = 0x1`: - First cluster: 26 writes for "Hello Deepak from Nielit!" ($\times 10 = 260$ total) - Then: reads at `0x1000_0008` (status polling) - Then: 4 writes at `0x1000_0000` (echo of PING characters)

Firmware String in ROM

In GTKWave when reading ROM during .data load (AUIPC + copy loop): - `cpu_mem_rdata` will show `0x6C6C6548` = little-endian "Hell" segment of the string

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
<code>make fw</code> fails: compiler not found	GCC toolchain not in PATH	<code>export PATH=\$PATH:/path/to/riscv-gcc/bin</code>
<code>firmware.elf</code> created but <code>rom.hex</code> wrong	<code>bin2hex.py</code> encoding error	Check Python3 is used; run <code>python3 fw/bin2hex.py fw/firmware.bin</code> manually
CPU traps immediately	Stack pointer initialization failed	Check <code>link.ld</code> <code>STACK_TOP</code> address matches PicoRV32 <code>STACKADDR</code> parameter
SB (byte store) writes 32 bits	Compiler generated word store	Add volatile <code>char*</code> cast explicitly: <code>UART_TX = c</code> where <code>UART_TX</code> is volatile <code>char*</code>
String appears garbled	ROM hex not updated	Run <code>make fw</code> again after modifying firmware
CPU loops forever before printing	<code>crt0.S</code> BSS clear corrupts UART reg	Ensure BSS region does not overlap UART address (it doesn't: SRAM is at <code>0x10000</code>)
Missing - <code>march=rv32i</code>	Division instruction generated	PicoRV32 has <code>ENABLE_DIV=0</code> — <code>rv32i</code> forbids compiler from emitting divide

Verify RISC-V ABI with objdump

`riscv32-unknown-elf-objdump -h fw/firmware.elf`

Check: - `.text` VMA = `0x00000000` (in ROM) - `.data` VMA = `0x00010000` (in SRAM) — but LMA = somewhere in ROM (copied by startup)

9. Summary

Topic	Detail
Toolchain	riscv32-unknown-elf-gcc with -march=rv32i -mabi=ilp32
crt0.S	Initializes stack to 0x0001_00FC, calls main(), traps on return
link.ld	ROM: 0x0-0x7FFF (32KB), SRAM: 0x10000-0x17FFF (8KB)
MMIO pattern	volatile pointers to fixed addresses for hardware access
rom.hex	4-byte-wide Intel-compatible hex; loaded by \$readmemh in sim
Firmware size	~167 bytes (42 instructions) — extremely compact
Functionality	Print “Hello Deepak from Nielit!” × 10, then echo received bytes
Verification	make soc → TEST PASSED (PING echoed)

Understanding firmware is essential for any SoC verification engineer. The hex values you see on `cpu_mem_rdata` during instruction fetch are the RISC-V opcodes directly compiled from this C code.

Previous: [Sub-module 6 — AXI + UART Integration](#)

Next: [Sub-module 8 — Full SoC Integration](#)